

1 LED マトリクスでの BPM 表示（変更後）

1.0.1 ファイル構成

表 1 BPM 表示機能のファイル構成

ファイル名	概要
Display.ino	WiFi 接続・UDP 受信開始の管理
Display.cpp	表示文字の独自フォント定義, LED マトリクスへの描画処理, UDP 受信処理
Display.h	定数定義, 関数宣言, 外部参照

ファイル構成について説明する。また、表 1 に BPM 表示機能のファイル構成を示している。はじめに、Display.ino ファイルでは、表 1 に示すように WiFi 接続と UDP 接続を行う。今回、指揮機と中継機間の通信形式としては、高速通信を目的とし、UDP 形式で情報を受け取る。また、WiFi 接続の前に `WiFi.config(localIP, gateway, subnet)` を呼び出し、指揮デバイスの送信先 IP アドレスと一致する静的 IP を設定する。

次に、Display.cpp ファイルでは、表示文字の独自フォント定義、マトリクスへの描画処理、そして通信処理を行う。また、Display.h ファイルでは定数の定義と関数宣言、独自フォント定義などを行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 を以下に示す。

```
{1,1,1,  
1,0,1,  
1,0,1,  
1,0,1,  
1,1,1}
```

また、LED マトリクスの描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3×5 のフォント配列を用いてフレームバッファに書き込むことで行う。

1.0.2 オブジェクト図 (クラス図)

LEDMatrix プログラムのクラス図は以下の図 1 の通りである。通信開始を行う Display.ino、通信受信処理と LED マトリクスへの描画処理を行う Display.cpp、および関数宣言やフォント定義を行う Display.h で構成される。各モジュール間の関係について、Display.ino から Display.cpp への関係は、表示処理を利用する依存関係である。また、Display.cpp から Display.h への関係は、関数宣言およびフォント定義を参照する依存関係である。

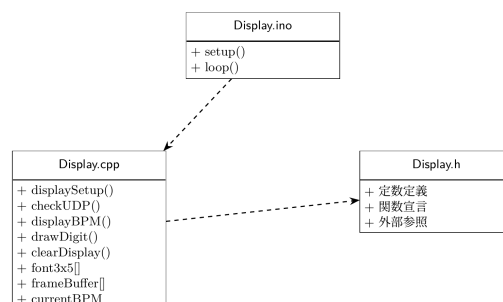


図 1 LEDMatrix のクラス図

1.1 関数設計

表 2 に今回用いる主な処理を行う関数を示している。以降では、各関数の具体的な動作を説明する。

はじめに、displaySetup 関数について説明する。この関数は、表 2 に示すように、LEDMatrix の初期化を行い、引数や戻り値はない。matrix.begin() を呼び出した後、消灯フレームを一度描画してから 200 ms 待機する。これは、begin() 直後の最初の loadFrame() は反映されない場合があるための対策である。

次に、checkUDP 関数について説明する。この関数は表 2 に示すように、中継機からのパケット確認と BPM 情報の値確認を行い、引数や戻り値はない。まず、中継機からのパケットを Udp.parsePacket() を用いて監視する。パケットが届いていることを確認できれば、uint8_t buffer[2] としてバイナリ形式で 2 バイトを読み取る。buffer[0] をヘッダ文字として判定し、ヘッダが'E' の場合は clearDisplay() を呼び出して LED を消灯する。また、ヘッダが'B' の場合は buffer[1] の段階番号を bpmTable[] で参照して BPM 値に変換し、値が更新されていれば描画更新処理を行う。

次に、displayBPM 関数について説明する。この関数は表 2 に示すように、BPM 数値のマトリクス描画を行い、引数としては、checkUDP 関数から保持した数値を用いる。まず、BPM 値を各桁ごとに分割し、drawDigit() を用いてフレームバッファに描画する。そして、uint32_t[3] 形式にビットパックした後、matrix.loadFrame() で描画を行う。

次に、drawDigit 関数について説明する。この関数は、表 2 に示すように、一桁の数字をフレームバッファに描画をし、引数としては描画する数字とフレームバッファ上の横方向開始位置を取る。まず、引数の数字が 0 から 9 の範囲内であるかを確認する。次に、縦方向の中央寄せオフセットを算出して配置する。その後、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取り、フレームバッファの指定位置に書き込む。

最後に、clearDisplay 関数について説明する。この関数は、表 2 に示すように、LEDMatrix を全消灯させる処理を行い、引数や戻り値はない。BPM 値を初期値にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。

表 2 主要な処理を行う関数一覧

関数名	概要	引数	戻り値
displaySetup	LED マトリクスの初期化	なし	なし
checkUDP	UDP パケットの確認、BPM 更新・消灯処理	なし	なし
displayBPM	BPM 数値を LED マトリクスに描画	int bpm	なし
drawDigit	1 桁の数字をフレームバッファに描画	int digit, int x_offset	なし
clearDisplay	LED マトリクスを全消灯	なし	なし